# LECTURE 7: RECOMMENDER SYSTEMS

DS 8001

# Table of Contents


1. Background
2. Content-based Recommender Systems
3. Collaborative Filtering
4. Evaluation
5. The Netflix Prize and Learned Interpolation Weights
6. Latent Factor Models


Toronto Metropolitan University

# RECOMMENDER SYSTEMS

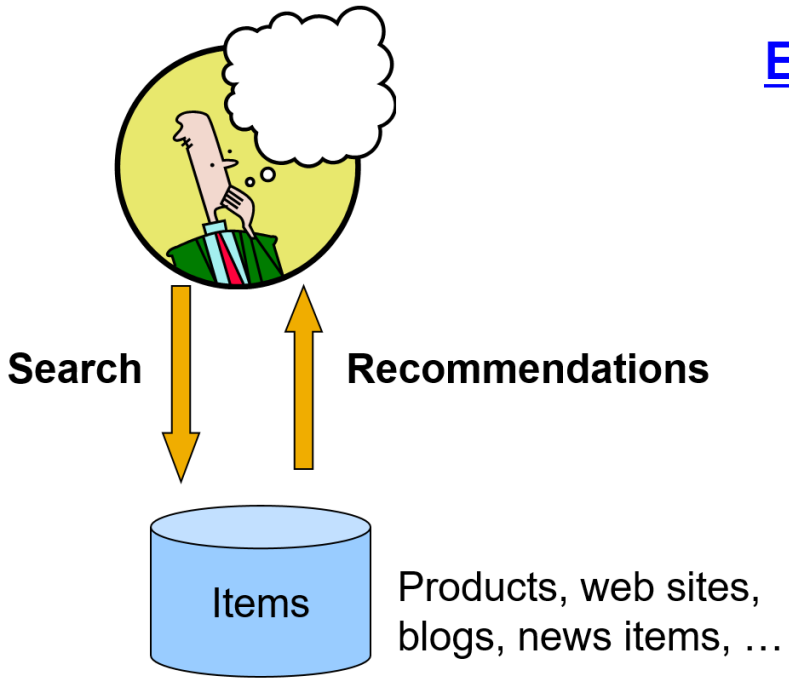
- A *recommender system* predicts each user's interest in each item and returns a short personalized list
- *Recommended reading*:

  Mining of Massive Datasets *by* Leskovec, Rajaraman, Ullman

  Ch 9: Recommendation Systems [Link]

- *Recommender Systems Example*:
  - Customer X
    - ✓ Buys Metallica CD
    - ✓ Buys Megadeth CD
  - Customer Y
    - ✓ Does search on Metallica
    - ✓ Recommender system suggests Megadeth from data collected about customer X

# SEARCH VS. RECOMMENDATIONS




**Examples:**

amazon.com
PANDORA
StumbleUpon
del.icio.us
NETFLIX
movielens
helping you find the *right* movies
last·fm
the social music revolution
Google News
YouTube
XBOX LIVE

J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets, http://www.mmds.org

*Search* is a *pull* interaction (the user states a need); *recommendation* is a *push* interaction (the system infers the need from past behavior). The logos date the slide: StumbleUpon and del.icio.us no longer operate.

# From Scarcity to Abundance

- Shelf space is a scarce commodity for traditional retailers
  - Retailers can only stock a certain number of products, usually those that are most popular or profitable
  - Also: TV networks, movie theaters etc can only showcase a limited # of shows/movies
- Web enables near-zero-cost dissemination of information about products
  - From scarcity to abundance
- More choice necessitates better filters
  - Recommendation engines
  - The *long tail*: online, the many rarely demanded items together outsell the few blockbusters – if a filter can surface them
  - How Into Thin Air made Touching the Void a bestseller: http://www.wired.com/wired/archive/12.10/tail.html

Toronto Metropolitan University

# Types of Recommendations

- Editorial and hand curated → *not* personalized
  - List of favorites
  - Lists of "essential" items
- Simple aggregates → same list for everyone
  - Top 10, Most Popular, Recent Uploads
- Tailored to individual users → the subject of this lecture
  - Amazon, Netflix,...

# Formal Model

- $X$ = Set of Customers (users)
- $S$ = Set of Items

- Utility function $u : X \times S \rightarrow \mathcal{R}$
  - $\mathcal{R}$ = set of possible ratings
  - $\mathcal{R}$ is a totally ordered set (so "better" is well defined)
  - E.g., 0-5 stars, real number in [0,1]
  - $u$ is only *partially* observed: we know $u(x, i)$ for a small set of pairs and must infer the rest
  - We reserve $R$ for the rating (utility) *matrix*, and write $(x, i) \in R$ for its observed entries

# Utility Matrix

Rows = users, columns = items, entries = known utilities (here rescaled to $[0, 1]$), "–" = *unknown*:

| | Avatar | LOTR | Matrix | Pirates |
|---|---|---|---|---|
| Alice | 1 | - | 0.2 | - |
| Bob | - | 0.5 | - | 0.3 |
| Carol | 0.2 | - | 1 | - |
| David | - | - | - | 0.4 |

In a real system almost every cell is "–". The task is to fill in the blanks and return the largest predicted entries of each row.

# KEY PROBLEMS

(1) Gathering "known" ratings for matrix

- *Explicit*: ask people to rate items – doesn't work well in practice: people can't be bothered, and those who do rate are a self-selected, biased sample
- *Implicit*: learn ratings from user actions, e.g., purchase implies high rating. What about low ratings? Inaction is ambiguous: a user who never bought an item may dislike it – or may simply never have seen it

(2) Extrapolate unknown ratings from the known ones

- Mainly interested in high unknown ratings
  - ✓ We are not interested in knowing what you do not like but what you like

(3) Evaluating extrapolation methods

- How to measure success/performance of recommendation methods?

# EXTRAPOLATING UTILITIES

- *Key problem*: Utility matrix $R$ is sparse
  - Most people have not rated most items

    E.g., Netflix Prize: $10^8$ ratings in a 480,000 $\times$ 17,770 matrix ($8.5 \times 10^9$ cells) $\rightarrow$ about 1% observed
  - *Cold start*:
    - ✓ New items have no ratings
    - ✓ New users have no history

- Three main approaches to recommender systems:
  1) Content-based
  2) Collaborative
  3) Latent factor-based

# CONTENT-BASED RECOMMENDATIONS

- *Main idea*: Recommend items to customer $x$ similar to previous items rated highly by $x$

  Similarity is measured between *item descriptions* – no other user's ratings are used

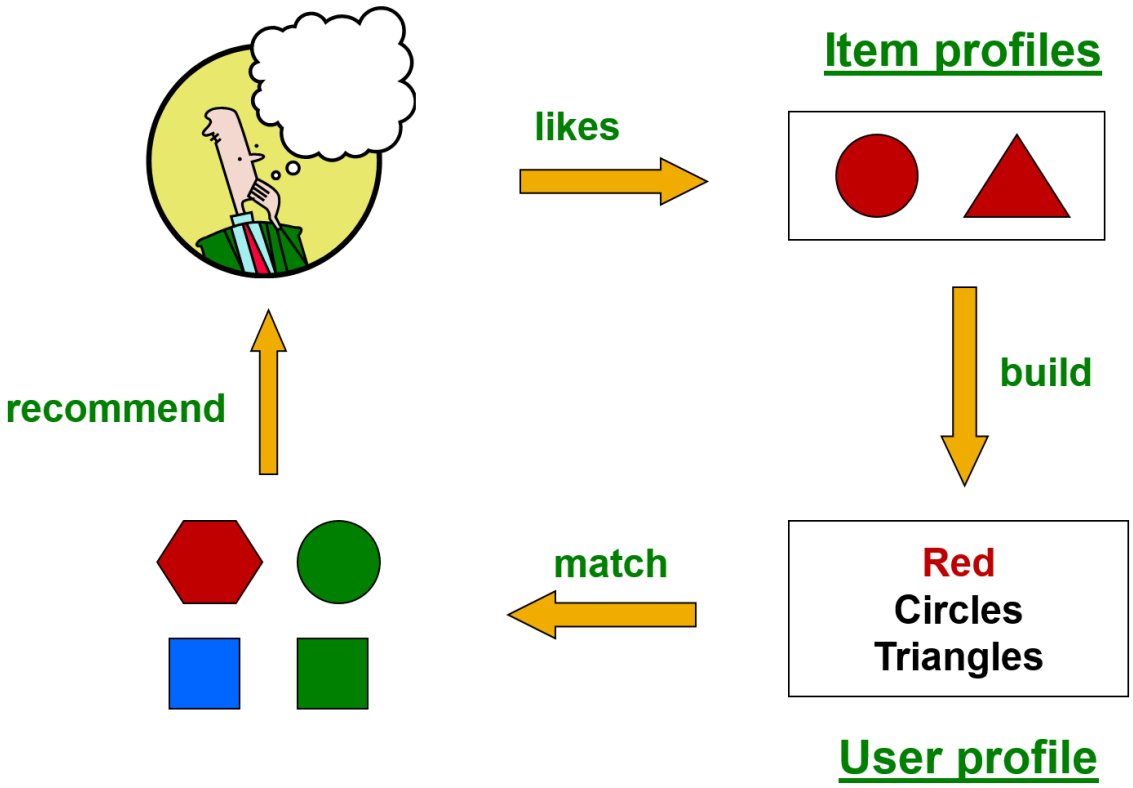
- *Example*:
  - Movie recommendations

    $\Rightarrow$ Recommend movies with same actor(s), director, genre, ...
  - Websites, blogs, news

    $\Rightarrow$ Recommend other sites with "similar" content

- For each item, create an *item profile* – a vector $\mathbf{i}$ in a common feature space
  - Movies: title, genre, year, actors, director, ...
  - Text: set of "important" words in document, usually picked by TF-IDF (Term Frequency $\times$ Inverse Document Frequency; next slides)

# THE CONTENT-BASED PIPELINE




J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets, http://www.mmds.org

Clockwise: the user *likes* some items → each item carries a *profile* (color, shape) → averaging the liked profiles *builds* a user profile ("red, circles, triangles") → we *match* it against unseen items and *recommend* the closest ones.

# TF-IDF FOR ITEM PROFILES

- $f_{ij}$ = frequency of term (feature/word) $i$ in doc (item/document) $j$

$$TF_{ij} = \frac{f_{ij}}{\max_k f_{kj}}$$

*Note*: We normalize TF to discount for "longer" documents ($\max_k$ runs over the terms of the *same* document $j$; stop words are removed first)

- $n_i$ = number of documents that mention term $i$; $N$ = total number of docs

$$IDF_i = \log \frac{N}{n_i}$$

$\rightarrow$ a term occurring in every document scores $IDF = 0$; rare, distinctive terms score high

- *TF-IDF* score: $w_{ij} = TF_{ij} \times IDF_i$
- *Doc profile* = set of words with highest TF-IDF scores, together with their scores

Toronto Metropolitan University

# TF-IDF: A Worked Example

- Movie descriptions ($N = 3$ documents):
  - M1: "Adventure in the mountains and forests"
  - M2: "Dramatic adventure in the city"
  - M3: "Documentary adventure about forests and wildlife"
- TF-IDF profiles ($TF = 1$ for every term; $\log_2$): "adventure" is in all 3 documents, so $IDF = 0$ and it drops out; "forests" is in 2 ($\log_2 \frac{3}{2} = 0.58$); every other term is in 1 ($\log_2 3 = 1.58$):

```
m1_profile={"forests":0.58, "mountains":1.58}
m2_profile={"city":1.58, "dramatic":1.58}
m3_profile={"documentary":1.58, "forests":0.58, "wildlife":1.58}
```

- Vectorization (*term order*: adventure, city, documentary, dramatic, forests, mountains, wildlife):

```
m1_vec = [0.0, 0.0,  0.0,  0.0,  0.58, 1.58, 0.0 ]
m2_vec = [0.0, 1.58, 0.0,  1.58, 0.0,  0.0,  0.0 ]
m3_vec = [0.0, 0.0,  1.58, 0.0,  0.58, 0.0,  1.58]
```

- $\cos(m_1, m_3) = \frac{0.34}{1.68 \times 2.31} = 0.09 > \cos(m_1, m_2) = 0 \rightarrow$ a user who liked M1 is shown M3: M1 shares only "adventure" with M2, and a word found everywhere carries no information

# User Profiles and Prediction

- User profile **x**: one vector in the *same* feature space as the items
  - Weighted average of rated item profiles
  - Variation: weight by difference from the user's average rating

    $\rightarrow$ needed because a user who rates *everything* 5 stars would otherwise get a profile matching every item
- Prediction heuristic:
  - Given user profile $x$ and item profile $i$, estimate

$$\hat{u}(x,i) = cos(x,i) = \frac{x \cdot i}{||x|| \cdot ||i||}$$

  - Recommend the unseen items with the largest $\cos(x,i)$

# Content-based Approach: Pros and Cons

*Pros*

- No need for data on other users
  - No item cold-start or sparsity problems
- Able to recommend to users with unique tastes
- Able to recommend new & unpopular items
  - No first-rater problem
- Able to provide explanations
  - Can list the content-features that caused an item to be recommended

*Cons*

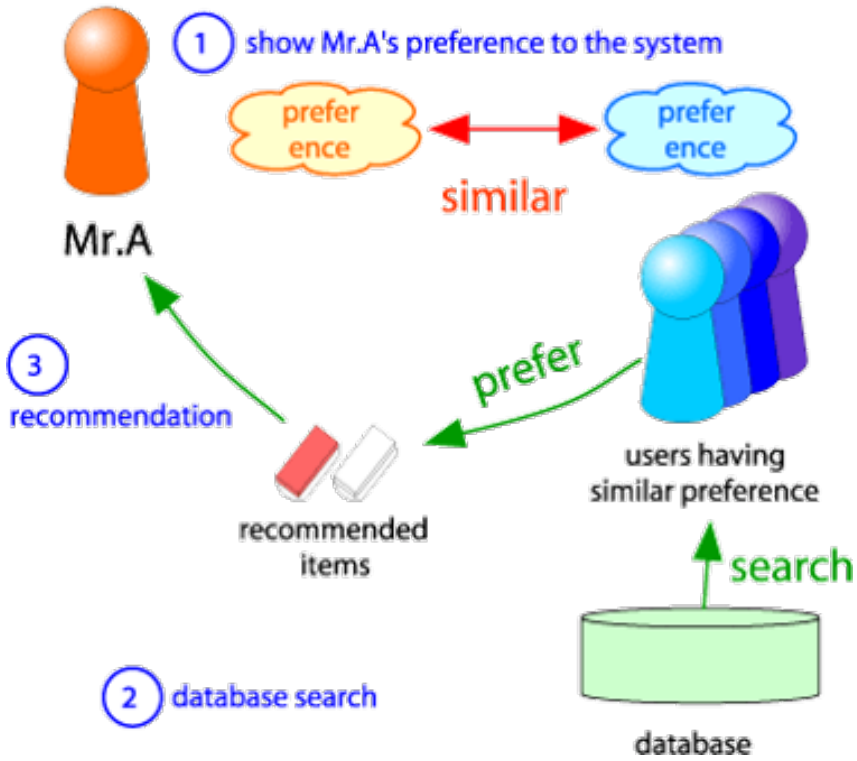
- Finding the appropriate features is hard
  - E.g., images, movies, music
- Recommendations for new users
  - How to build a user profile?
- Overspecialization
  - Never recommends items outside user's content profile
  - People might have multiple interests
  - Unable to exploit quality judgments of other users

$\rightarrow$ content-based methods survive a *new item* but not a *new user*: cold start is only half solved.

# COLLABORATIVE FILTERING (USER-USER)

- Consider user $x$
- Find set $N$ of other users whose ratings are "similar" to $x$'s ratings

  $N$ is the *neighborhood* of $x$; $k = |N|$ is a tuning parameter
- Estimate $x$'s ratings based on ratings of users in $N$
- No item content is used – only the rating matrix



# Finding Similar Users

- Let $r_x$ be the vector of user $x$'s ratings
  - $r_x, r_y$ as sets of *items rated*: $r_x = \{1, 4, 5\}$, $r_y = \{1, 3, 4\}$
- Jaccard similarity measure $= |r_x \cap r_y| / |r_x \cup r_y|$
  - *Problem*: Ignores the value of the rating
- Cosine similarity measure

$$sim(x, y) = cos(r_x, r_y) = \frac{r_x \cdot r_y}{||r_x|| \cdot ||r_y||}$$

  - $r_x, r_y$ as vectors, unrated items set to 0: $r_x = (1, 0, 0, 1, 3)$, $r_y = (1, 0, 2, 2, 0)$
  - *Problem*: Treats missing ratings as "negative" (like a 0-star item)
- Pearson correlation coefficient

  $S_{xy}$ = items rated by both users $x$ and $y$

$$sim(x, y) = \frac{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)(r_{ys} - \bar{r}_y)}{\sqrt{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)^2} \sqrt{\sum_{s \in S_{xy}} (r_{ys} - \bar{r}_y)^2}}$$

  ($\bar{r}_x, \bar{r}_y$: avg rating of $x, y$) $\rightarrow$ removes "harsh" vs. "generous" rating styles

# Similarity Metric

- Rows: users $A$–$D$; columns: 3 Harry Potter films, Twilight, 3 Star Wars films
- Intuitively we want: $sim(A, B) > sim(A, C)$
- *Jaccard*: $1/5 < 2/4$ – wrong order
- *Cosine*: $0.380 > 0.322$ – still wrong; considers missing ratings as "negative"

| | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
|---|---|---|---|---|---|---|---|
| $A$ | 4 | | | 5 | 1 | | |
| $B$ | 5 | 5 | 4 | | | | |
| $C$ | | | | 2 | 4 | 5 | |
| $D$ | | 3 | | | | | 3 |

| | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
|---|---|---|---|---|---|---|---|
| $A$ | $2/3$ | | | $5/3$ | $-7/3$ | | |
| $B$ | $1/3$ | $1/3$ | $-2/3$ | | | | |
| $C$ | | | | $-5/3$ | $1/3$ | $4/3$ | |
| $D$ | | 0 | | | | | 0 |

- *Solution*: subtract the row mean ($A$ averages $10/3$, so $4 \to 2/3$)
  $\Rightarrow$ sim(A,B) vs sim(A,C): $0.092 > -0.559$
- Notice cosine similarity is Pearson correlation when data is centered at 0: a blank now counts as "average"; the $S_{xy}$-only formula gives $1 > -0.73$: same order

# Rating Predictions

From similarity metric to recommendations:

- Let $r_x$ be the vector of user $x$'s ratings; $r_{xi}$ is $x$'s rating for item $i$ and $\hat{r}_{xi}$ our *prediction* of it
- Let $N$ be the set of $k = |N|$ users most similar to $x$ who have rated item $i$
- Prediction for item $i$ of user $x$:
  - *Option 1*: $\hat{r}_{xi} = \frac{1}{k} \sum_{y \in N} r_{yi}$ (plain average)
  - *Option 2*: $\hat{r}_{xi} = \frac{\sum_{y \in N} s_{xy} \cdot r_{yi}}{\sum_{y \in N} s_{xy}}$ $\rightarrow s_{xy} = sim(x, y)$ (similarity-weighted)
  - Other options
- Many other tricks possible... e.g. keep only neighbors with $s_{xy} > 0$, since negative weights make the denominator meaningless

Toronto Metropolitan University

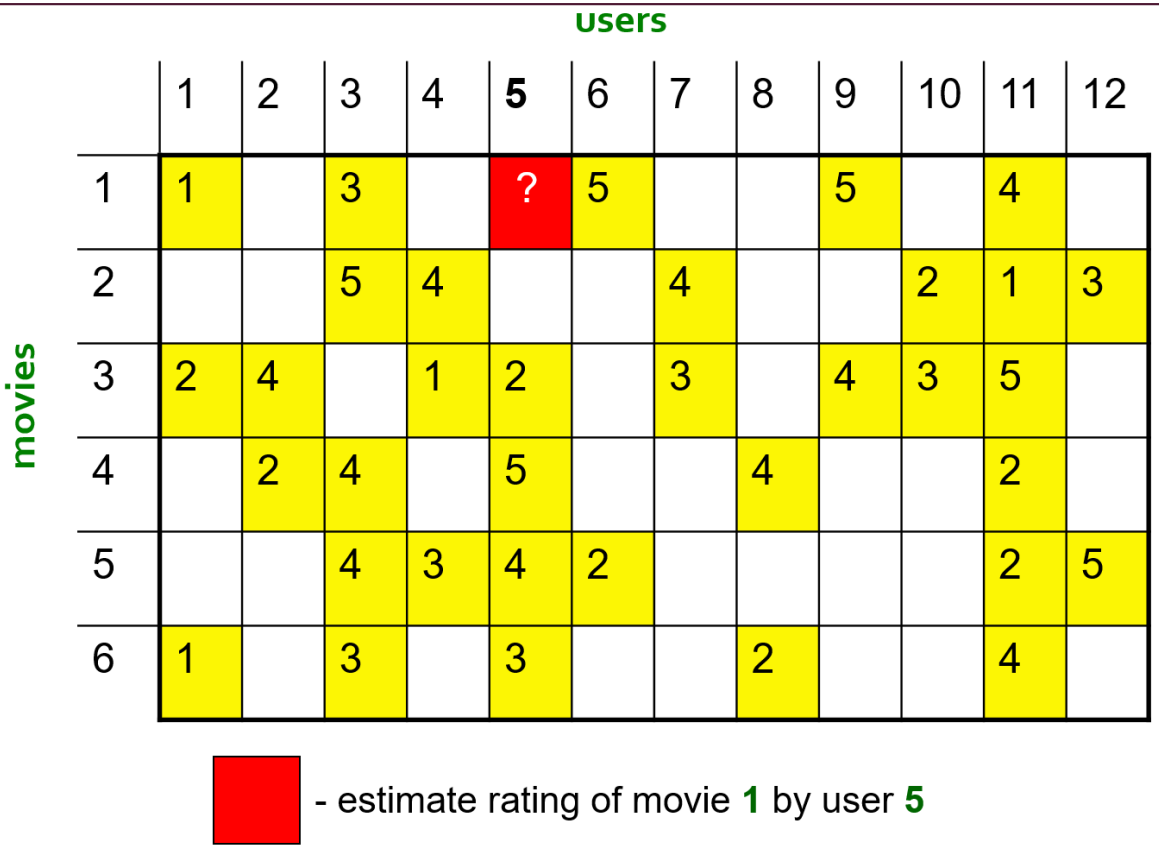
# ITEM-ITEM COLLABORATIVE FILTERING

- So far: *User-user* collaborative filtering
- Another view: *Item-item*
  - For item $i$, find other similar items (similar = rated alike by the same users, *not* similar in content)
  - Estimate rating for item $i$ based on ratings for similar items
  - Can use same similarity metrics and prediction functions as in user-user model

$$\hat{r}_{xi} = \frac{\sum_{j \in N(i;x)} s_{ij} \cdot r_{xj}}{\sum_{j \in N(i;x)} s_{ij}}$$

    - ✓ $s_{ij}$: similarity of items $i$ and $j$ (centered cosine between columns of $R$)
    - ✓ $r_{xj}$: rating of user $x$ on item $j$
    - ✓ $N(i;x)$: set of the $k$ items rated by $x$ that are most similar to $i$

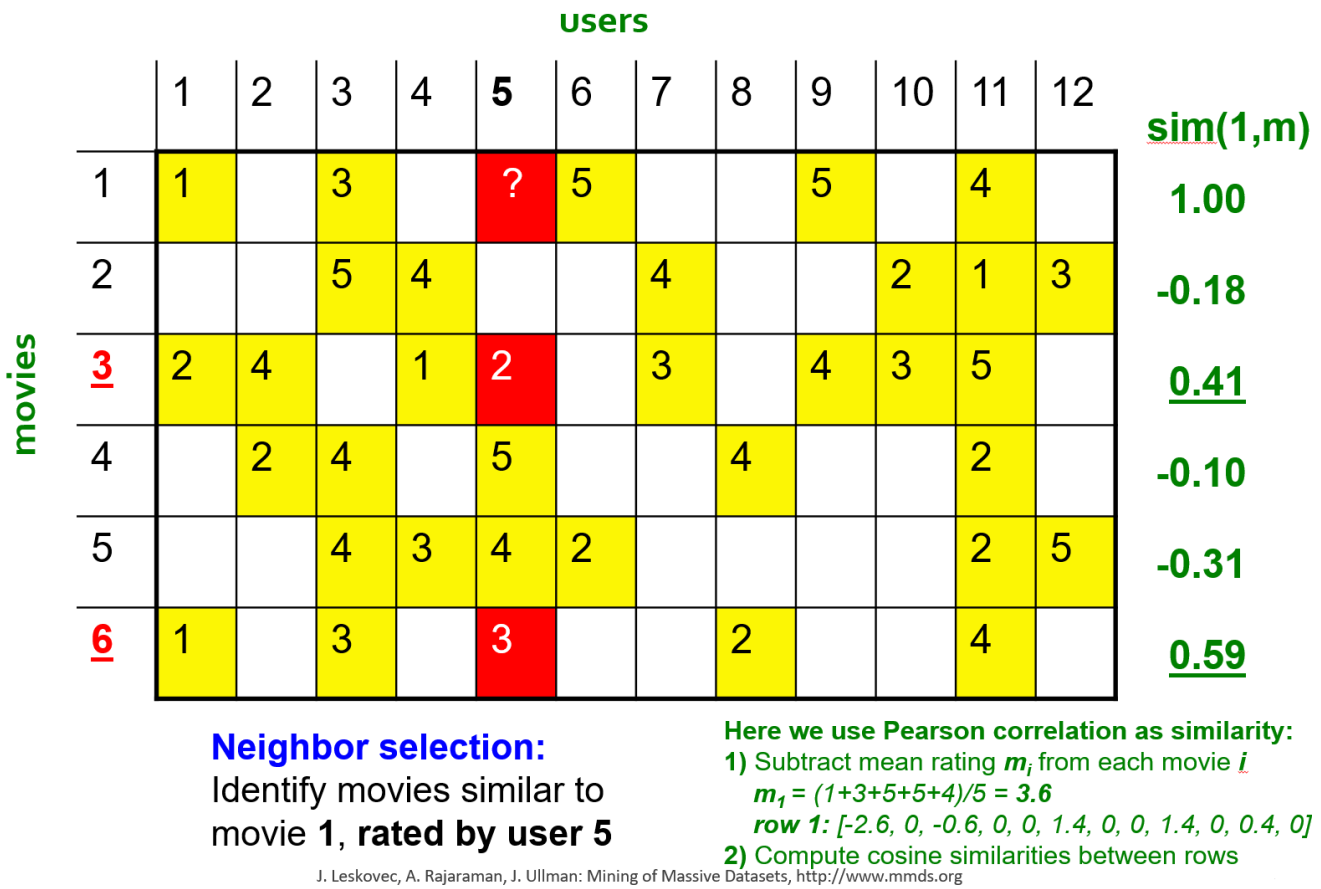
# ITEM-ITEM CF (1/3): THE TARGET CELL




J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets, http://www.mmds.org

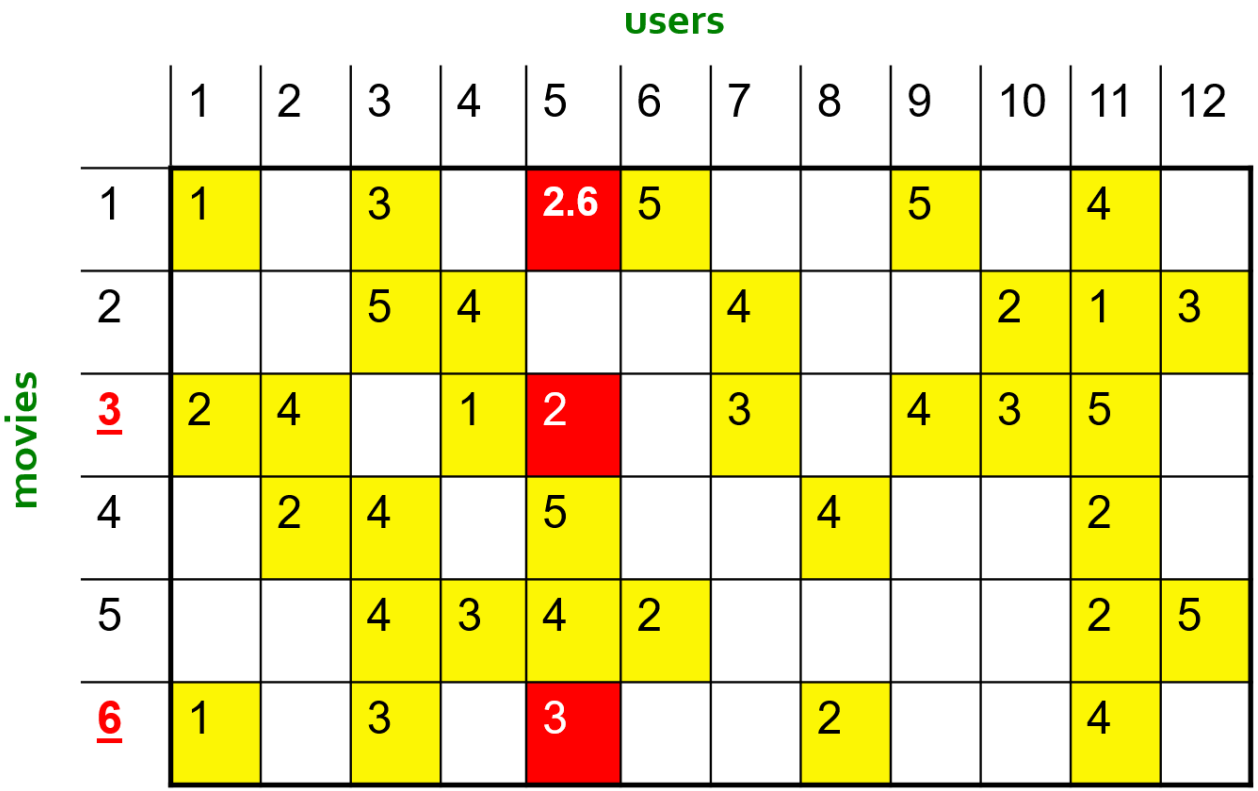
*Rows are movies, columns are users* – the transpose of the utility matrix shown earlier. Yellow cells are observed ratings on a 1–5 scale; blank cells are unknown. Goal: predict $\hat{r}_{xi}$ for user $x = 5$ and movie $i = 1$ (red cell), using the $k = |N| = 2$ movies most similar to movie 1 *among those user 5 has already rated.*

# ITEM-ITEM CF (2/3): NEIGHBOR SELECTION




Each row is centered by its own mean ($m_1 = 3.6$), then cosine similarities to row 1 are computed. User 5 rated movies $3, 4, 5, 6$, with similarities $0.41, -0.10, -0.31, 0.59$: the two neighbors are movies **6** and **3**, with weights $s_{1,3} = 0.41$ and $s_{1,6} = 0.59$; user 5 rated these movies 2 and 3 respectively.

# ITEM-ITEM CF (3/3): PREDICTION




**Predict by taking weighted average:**

$r_{1.5}$ = (0.41*2 + 0.59*3) / (0.41+0.59) = 2.6

$$r_{ix} = \frac{\sum_{j \in N(i;x)} s_{ij} \cdot r_{jx}}{\sum s_{ij}}$$

J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets, http://www.mmds.org

$\hat{r}_{5,1} = \frac{0.41 \times 2 + 0.59 \times 3}{0.41 + 0.59} = 2.59 \approx 2.6$. *Caution*: the figure writes subscripts item-first ($r_{ix}$); these slides write them user-first ($r_{xi}$).

# CF: Common Practice

- Define similarity $s_{ij}$ of items $i$ and $j$
- Select $k$ nearest neighbors $N(i;x)$
  - Items most similar to $i$, that were rated by $x$
- Estimate rating $\hat{r}_{xi}$ as the weighted average of *deviations from a baseline*:

$$\hat{r}_{xi} = b_{xi} + \frac{\sum_{j \in N(i;x)} s_{ij} \cdot (r_{xj} - b_{xj})}{\sum_{j \in N(i;x)} s_{ij}}$$

  - $b_{xi}$: baseline estimate for $r_{xi} \rightarrow b_{xi} = \mu + b_x + b_i$
    - ✓ $\mu$ = overall mean movie rating
    - ✓ $b_x$ = rating deviation of user $x$ = (avg rating of user $x$) - $\mu$
    - ✓ $b_i$ = rating deviation of movie $i$ = (avg rating of movie $i$) $-$ $\mu$
    - ✓ Subtracting $b_{xj}$ first means the neighbors contribute only what is *surprising* about their ratings

Toronto Metropolitan University

# ITEM-ITEM VS. USER-USER

A second toy utility matrix (its values differ from the earlier one):

| | Avatar | LOTR | Matrix | Pirates |
|---|---|---|---|---|
| Alice | 1 | - | 0.8 | - |
| Bob | - | 0.5 | - | 0.3 |
| Carol | 0.9 | - | 1 | 0.8 |
| David | - | - | 1 | 0.4 |

- *Concept check*: predict David's rating for Avatar (raw cosine: with 1–3 ratings per user or item, centering leaves too little to compare)
  - ⇒ *Item-item*: of the items David rated, Matrix is closest to Avatar (Alice and Carol rate the two alike: cosine 0.78 vs. 0.57 for Pirates) → predict ≈ his Matrix rating, i.e. high
  - ⇒ *User-user*: only Alice and Carol rated Avatar; both resemble David (cosine 0.58 and 0.78) but share just one or two items with him → predict ≈ 0.9–1; resting on such thin overlaps, this is the more fragile route
- In practice, it has been observed that item-item often works better than user-user.
  - ⇒ Items are simpler, users have multiple tastes – and an item's rating pattern is more stable over time

# Pros/Cons of Collaborative Filtering

- Pro: Works for any kind of item
  - No feature selection needed → the same code serves movies, songs and groceries
- Con: Cold start
  - Need enough users in the system to find a match
- Con: Sparsity
  - The user/ratings matrix is sparse
  - Hard to find users that have rated the same items
- Con: First rater
  - Cannot recommend an item that has not been previously rated
  - New items, esoteric items
- Con: Popularity bias
  - Cannot recommend items to someone with unique taste
  - Tends to recommend popular items → neglects the long tail we set out to reach

# Hybrid Methods

- Implement two or more different recommenders and combine predictions
  - Perhaps using a linear model whose weights are themselves fit on held-out data
- Add content-based methods to collaborative filtering
  - Item profiles for new item problem
  - Demographics to deal with new user problem
- *Concept check*: a brand-new film is added to the catalog with zero ratings. Which family can recommend it, and why?

  $\Rightarrow$ Only the content-based half: CF has no column of ratings to compute $s_{ij}$ from. This is precisely why production systems are hybrids.

# Tip: Add Data

- Leverage all the data
  - Don't try to reduce data size in an effort to make fancy algorithms work
  - Simple methods on large data are a strong baseline
- Add more data
  - e.g., in a Netflix movie rating prediction task, add IMDB data on genres
- More data beats better algorithms
  - Link
  - Read this as a *heuristic*, not a law: on the Netflix Prize the data were fixed and the whole 10% gain came from better models

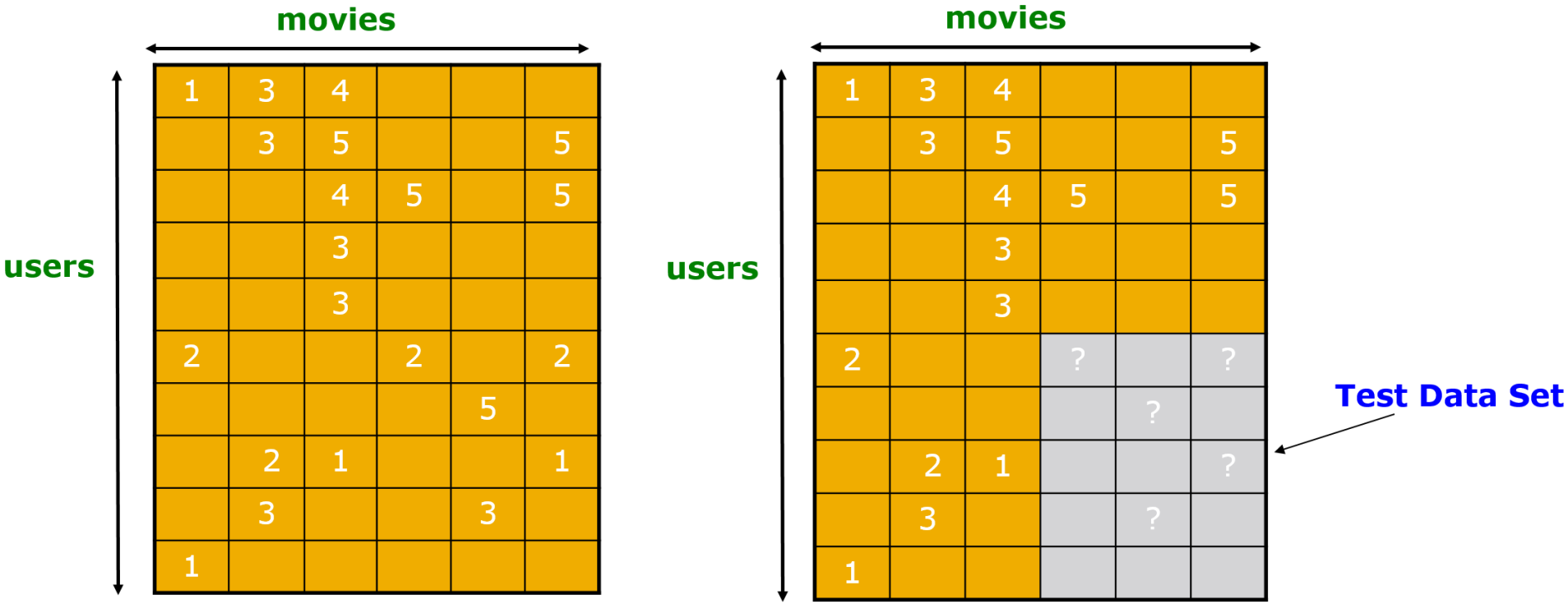
# Collaborative Filtering: Complexity

- Expensive step is finding $k$ most similar customers to one user: $O(|X|)$
- Too expensive to do at runtime
  - Could pre-compute
- Naive pre-computation for *all* users takes time $O(|X|^2)$
  - $X$... set of customers ($|X| \approx 4.8 \times 10^5$ for Netflix $\rightarrow$ over $10^{11}$ pairs)
- We already know how to do this!
  - Near-neighbor search in high dimensions (LSH: locality-sensitive hashing)
  - Clustering
  - Dimensionality reduction $\rightarrow$ the route taken by latent factor models, later in this lecture

Toronto Metropolitan University

# EVALUATION

How to pick the evaluation set?




We can only score a prediction where the truth is known, so we *hide* some of the observed ratings (grey "?" cells), fit the model on the rest, and compare predictions against the hidden values. Hiding a block of recent ratings, as Netflix did, mimics deployment better than hiding random cells – a model must predict the *future*, not interpolate the past.

# Evaluating Predictions

- Compare predictions with known ratings on the held-out set
  - Root-mean-square error (RMSE): $\sqrt{\frac{1}{|R|}\sum_{(x,i)\in R}(\hat{r}_{xi} - r_{xi})^2}$
    - ✓ $\hat{r}_{xi}$ is predicted, $r_{xi}$ is the true rating of $x$ on $i$, $|R|$ counts the held-out ratings
  - Precision at top 10 (precision@10): of the 10 items ranked highest for a user, the fraction the user actually liked
  - Rank correlation: Spearman's correlation between system's and user's complete rankings
- Another approach: 0/1 model – score the binary decision "recommend / do not recommend"
  - Coverage: Number of items/users for which system can make predictions
  - Precision: Accuracy of predictions = fraction of recommended items that were relevant
  - Receiver operating characteristic (ROC): Tradeoff curve between true-positive and false-positive rates as the "recommend" threshold is swept
- Production systems mostly report *ranking* metrics (recall@$k$, NDCG): users see a ranked list, not a number

Toronto Metropolitan University

# Problems with Error Measures

- Narrow focus on accuracy sometimes misses the point
  - Prediction Diversity – ten near-identical sequels can have a perfect RMSE and still be a bad list
  - Prediction Context – a gift purchase or a shared account is not evidence about the buyer's own taste
  - Order of predictions – RMSE weighs every held-out cell equally, but only the top of the list is ever seen
- In practice, we care only to predict high ratings:
  - RMSE might penalize a method that does well for high ratings and badly for others

# THE NETFLIX PRIZE

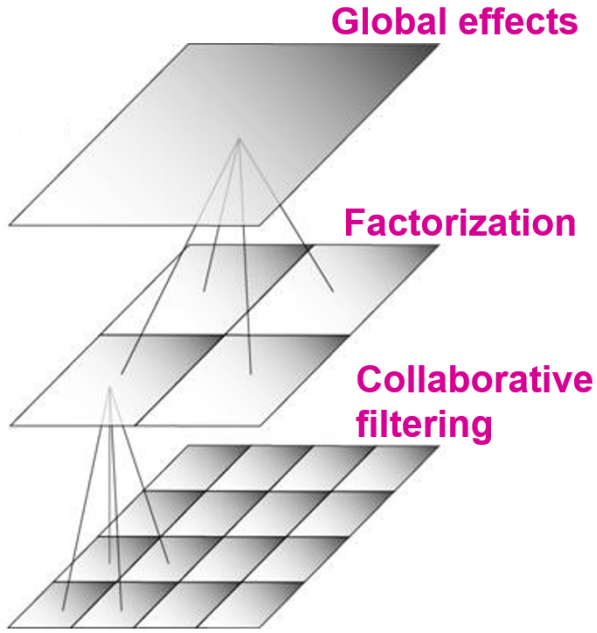
- Training data
  - 100 million ratings, 480,000 users, 17,770 movies
  - ratings collected between October 1998 and December 2005
- Test data
  - Last few ratings of each user (2.8 million), split at random into a public *quiz* half and a secret *test* half
  - Evaluation criterion: Root Mean Square Error (RMSE)

    $$RMSE = \sqrt{\frac{1}{|R|} \sum_{(x,i) \in R} (\hat{r}_{xi} - r_{xi})^2}$$

    the average must sit *inside* the root; $|R|$ = number of ratings scored
  - Netflix's own system, *Cinematch*, RMSE: 0.9514 on the quiz half (0.9525 on the test half)
- Competition (2006–2009)
  - 5,000+ teams worldwide
  - $1 million prize for 10% improvement on Cinematch

# BellKor Recommender System

- The winner of the Netflix Prize!
  More precisely: the 2009 grand prize went to *BellKor's Pragmatic Chaos* – BellKor blended with BigChaos and Pragmatic Theory – at test-half RMSE 0.8567 (the bar there was 0.8572). Netflix never deployed the full ensemble.
- Multi-scale modeling of the data: combine top level, "regional" modeling of the data, with a refined, local view
  - *Global*: Overall deviations of users/movies
  - *Factorization*: Addressing "regional" effects (i.e., not global/common across users or completely individual)
    - ✓ Factorization breaks down the matrix into smaller matrices that capture latent factors
  - *Collaborative filtering*: Extract local patterns




Coarse effects on top, finer ones below.

# Modeling Local & Global Effects

- *Global*:
  - Mean movie rating: 3.7 stars
  - The Sixth Sense is 0.5 stars above avg.
  - Joe rates 0.2 stars below avg.

    $\Rightarrow$ Baseline estimation $b_{xi} = 3.7 + 0.5 - 0.2$

    $\Rightarrow$ Joe will rate The Sixth Sense 4 stars

- *Local neighborhood (CF/NN)*:
  - Joe didn't like related movie Signs

    $\Rightarrow$ Final estimate = baseline + neighborhood correction

    $\Rightarrow$ Joe will rate The Sixth Sense 3.8 stars

# LIMITS OF SIMILARITY WEIGHTS

- Recall from "CF: Common Practice": In practice we get better estimates if we model deviations:

$$\hat{r}_{xi} = b_{xi} + \frac{\sum_{j \in N(i;x)} s_{ij} \cdot (r_{xj} - b_{xj})}{\sum_{j \in N(i;x)} s_{ij}}$$

$b_{xi} = \mu + b_x + b_i$: baseline estimate for $r_{xi}$

- *Problems/Issues*:
  1) Similarity measures are "arbitrary" – nothing says cosine is the right one
  2) Pairwise similarities neglect interdependencies among users and items: three near-duplicate movies each cast a full vote
  3) Taking a weighted average can be restricting – the weights are forced to sum to 1

*Solution*: Instead of $s_{ij}$ use $w_{ij}$ that we *learn* directly from data

# IDEA: INTERPOLATION WEIGHTS $w_{ij}$

- Use a weighted sum rather than weighted avg

$$\hat{r}_{xi} = b_{xi} + \sum_{j \in N(i;x)} w_{ij} \cdot (r_{xj} - b_{xj})$$

  - $N(i;x)$: set of movies rated by user $x$ that are similar to movie $i$
  - $w_{ij}$: the interpolation weights (some real number)
    We allow $\sum_{j \in N(i;x)} w_{ij} \neq 1$ – this is what lifts restriction 3) above
  - $w_{ij}$ models interaction between pairs of movies
    (it does not depend on user $x$) $\rightarrow$ every user who rated both $i$ and $j$ contributes evidence about $w_{ij}$
  - Weights $w_{ij}$ derived based on their role in the prediction; no use of an arbitrary similarity measure ($w_{ij} \neq s_{ij}$) – lifts restriction 1)
  - Explicitly account for interrelationships among the neighboring movies $\rightarrow$ two near-duplicate neighbors now *share* one weight instead of voting twice – lifts restriction 2)

Toronto Metropolitan University

# Recommendations via Optimization

- *Goal*: Make good recommendations, quantified by RMSE $\Rightarrow$ lower RMSE $\Rightarrow$ better recommendations
- Want to make good recommendations on items that user has not yet seen. We cannot score those directly, so we set $w$ to work well on the *known* (user, item) ratings and hope it also predicts the unknown ones
- *Idea*: Define an objective function and solve the optimization problem $\Rightarrow$ find $w_{ij}$ that minimize SSE on training data

$$J(w) = \sum_{(x,i)\in R} \left( \left[ b_{xi} + \sum_{j\in N(i;x)} w_{ij}(r_{xj} - b_{xj}) \right] - r_{xi} \right)^2$$

Remember:

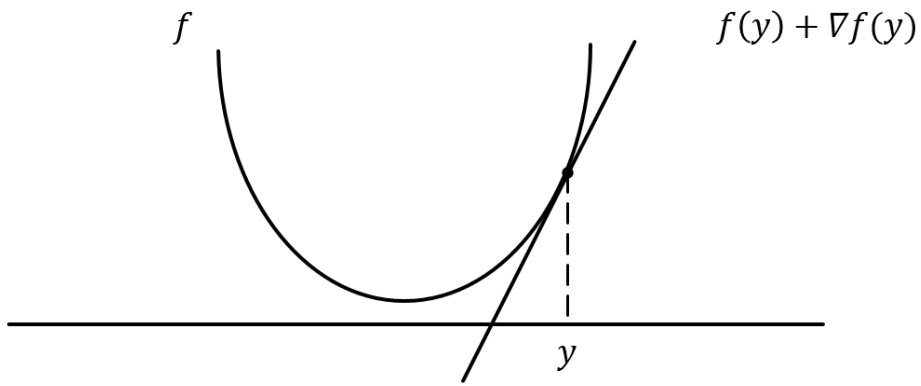
- $b_{xi} + \sum_{j\in N(i;x)} w_{ij}(r_{xj} - b_{xj})$ is the predicted rating $\hat{r}_{xi}$, $r_{xi}$ is the true rating
- $w$ is a vector of numbers, one entry per movie pair $(i,j)$
- SSE and RMSE are minimized by the same $w$, so we optimize SSE

Toronto Metropolitan University

# Review: Minimizing a Function

A simple way to minimize a function $f(y)$ (gradient descent)

- Compute the derivative $\nabla f$
- Start at some point $y$ and evaluate $\nabla f(y)$
- Make a step in the reverse direction of the gradient: $y \leftarrow y - \eta \nabla f(y)$ where $\eta > 0$ is the *step size* (learning rate): too large and the iterates diverge, too small and they crawl
- Repeat until convergence $\rightarrow$ finds a global minimum if $f$ is convex, otherwise a local one



# Interpolation Weights: Gradient Descent

- We have the optimization problem, now what?

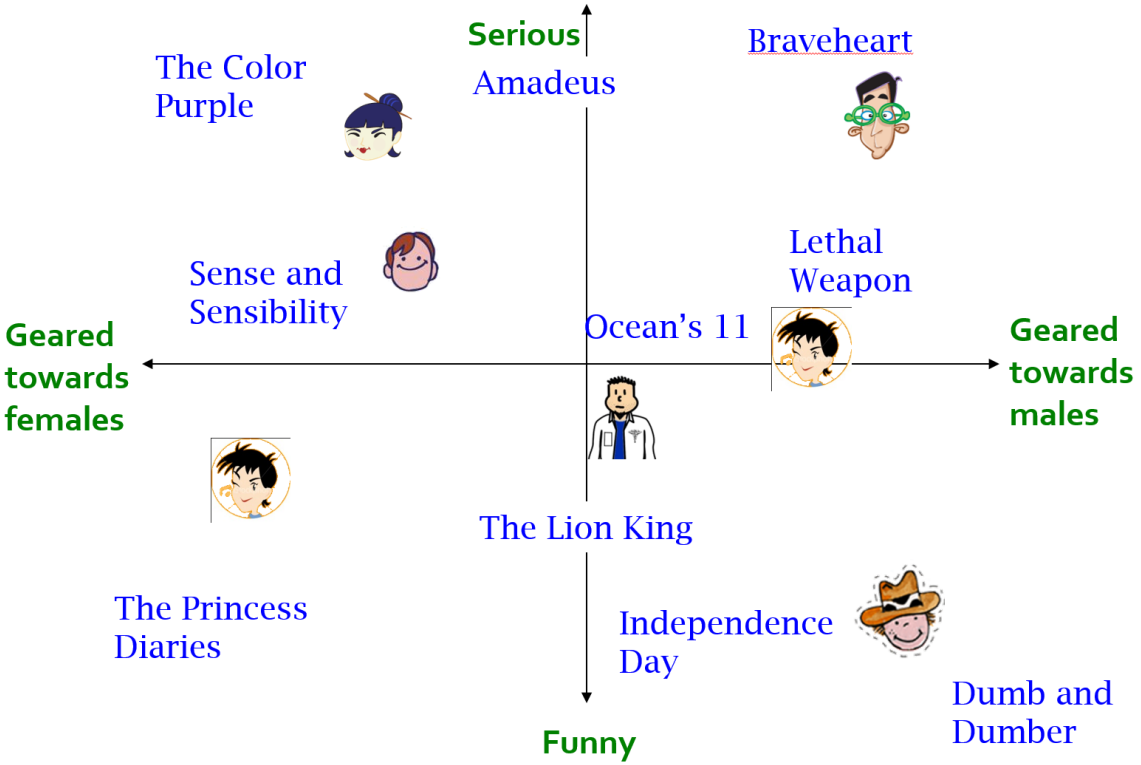
$$J(w) = \sum_{(x,i)\in R} \left( \left[ b_{xi} + \sum_{j \in N(i;x)} w_{ij}(r_{xj} - b_{xj}) \right] - r_{xi} \right)^2$$

- Gradient descent:
  - Iterate until convergence: $w \leftarrow w - \eta \nabla_w J$
    where $\nabla_w J$ is the gradient (derivative evaluated on data):

$$\frac{\partial J(w)}{\partial w_{ij}} = 2 \sum_{\substack{x\,:\,(x,i)\in R, \\ j \in N(i;x)}} \left( \left[ b_{xi} + \sum_{k \in N(i;x)} w_{ik}(r_{xk} - b_{xk}) \right] - r_{xi} \right) (r_{xj} - b_{xj})$$

  - Note: We fix movie $i$, sum only over the users $x$ who rated it and have $j$ among the neighbors $N(i;x)$; users without $j$ contribute nothing

# LATENT FACTOR MODELS FOR MOVIE RATINGS - 1




The model places every movie *and* every user at a point in the same $k$-dimensional space ($k = 2$ here). The axis *names* are supplied by a human after seeing where the movies landed: the algorithm returns unlabeled coordinates, and another run may produce different but equally valid axes.

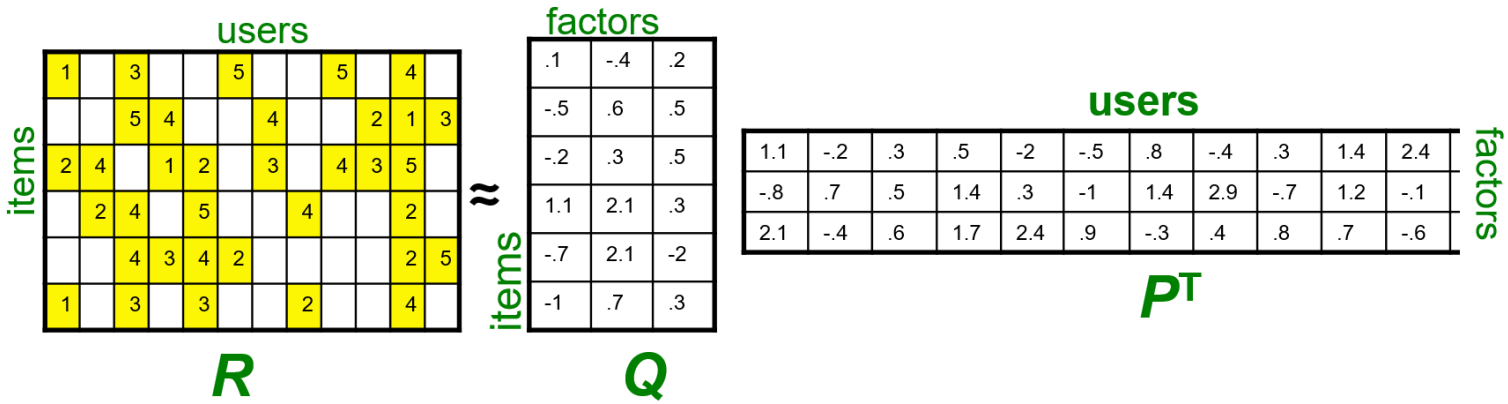
# LATENT FACTOR MODELS FOR MOVIE RATINGS - 2

- Latent factor models (e.g., Matrix Factorization) uncover latent factors from the user-item interactions (e.g., movie ratings) that explain observed preferences: these factors are not directly observed but are inferred from the data
- *Dimensionality Reduction*: a user's whole rating history is summarized by 2 main dimensions (latent factors): seriousness & audience orientation
- *Positioning*: items get a vector $q_i$ and users a vector $p_x$ in the *same* space; a user who likes "The Lion King" and "Independence Day" has a preference vector sitting between these two movies
- *Recommendations*: rank unseen items by proximity (dot product) to the user's vector; e.g. a fan of "Sense and Sensibility" is offered "The Color Purple": neighbors in factor space, though they share no actor or director
- This is the "regional" layer of the BellKor picture: coarser than one user's neighborhood, finer than a global average

# LATENT FACTOR MODELS

- "SVD" on Netflix data: $R \approx Q \cdot P^T$

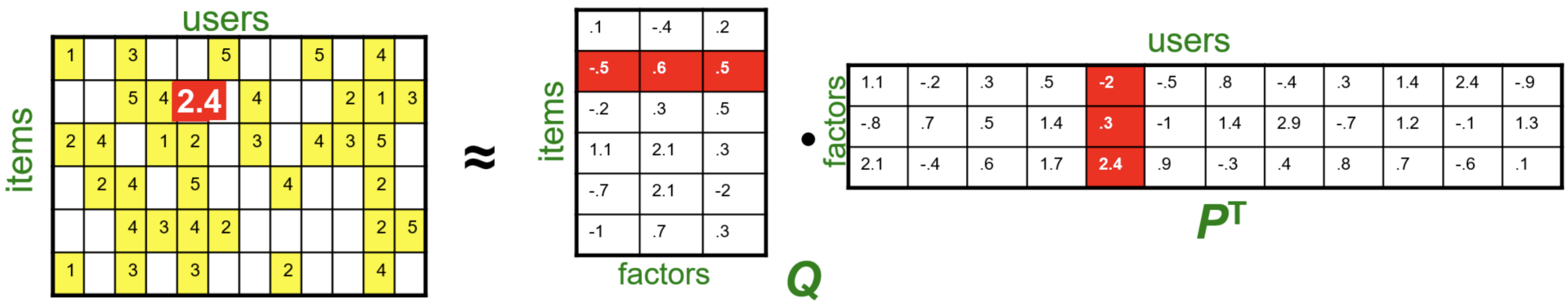
  Remember: SVD $\rightarrow A = U \Sigma V^T$




- $Q$ is $|S| \times k$, $P$ is $|X| \times k$: we store $k(|X| + |S|)$ numbers instead of $|X|\,|S|$ (here $R$ is items $\times$ users, as in the figure)
- For now let's assume we can approximate the rating matrix $R$ as a product of "thin" $Q \cdot P^T$
  - $R$ has missing entries but let's ignore that for now!
    - ✓ Basically, we want the reconstruction error to be small on known ratings and we don't care about the values on the missing ones

# RATINGS AS PRODUCTS OF FACTORS

- How to estimate the missing rating of user $x$ for item $i$?



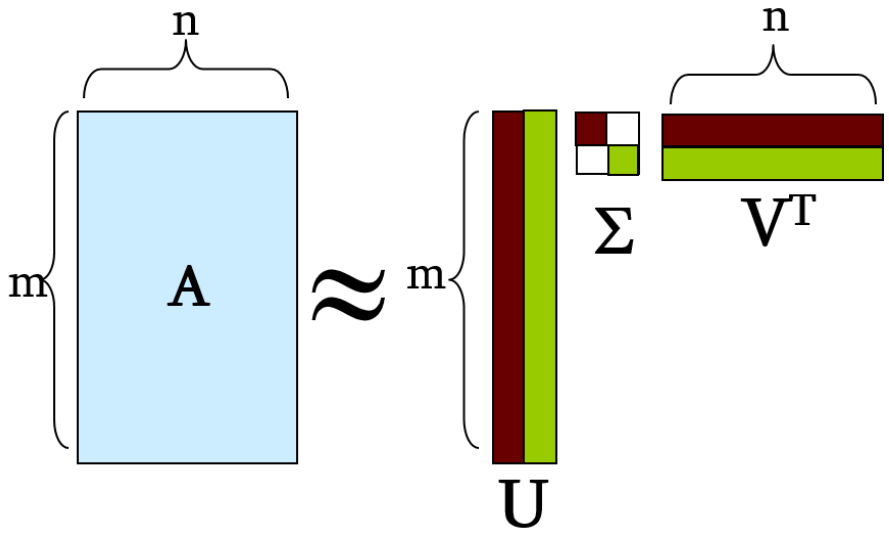

*Estimated rating*:

$$\hat{r}_{xi} = \mathbf{q}_i \cdot \mathbf{p}_x = \sum_f q_{if} \cdot p_{xf} = (-0.5)(-2) + (0.6)(0.3) + (0.5)(2.4) \approx 2.4$$

The red row $\mathbf{q}_i$ of $\mathbf{Q}$ (item $i$'s factors) $= (-0.5, 0.6, 0.5)$ and red column $\mathbf{p}_x$ of $\mathbf{P}^\mathsf{T}$ (user $x$'s factors) $= (-2, 0.3, 2.4)$ multiply out to the red cell of $R$.

# SVD Recap (1/2)

- Remember SVD: $A = U \cdot \Sigma \cdot V^\intercal$
  - $A$: Input data matrix ($m \times n$)
  - $U$: Left singular vecs
  - $V$: Right singular vecs
  - $\Sigma$: diagonal matrix of Singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$




Keeping only the top $k$ singular values gives the "thin" approximation drawn here.

- In our case:
  - matching $R \approx Q \cdot P^T$ term by term
    - ✓ $A = R$
    - ✓ $Q = U$
    - ✓ $P^T = \Sigma V^T$

# SVD Recap (2/2)

- SVD gives minimum reconstruction error (SSE) *among all rank-k matrices* (Eckart–Young):

$$\min_{U,\Sigma,V\,:\,\text{rank } k} \sum_{i,j} \left(A_{ij} - [U\Sigma V^T]_{ij}\right)^2$$

  Without the rank restriction the statement is empty: the full SVD reproduces $A$ exactly

- SSE and RMSE are monotonically related: $RMSE = \frac{1}{c}\sqrt{SSE}$ with $c = \sqrt{|R|}$, so minimizing SSE also minimizes RMSE
- Complication: The SVD error term sums over *all* entries (no-rating is interpreted as zero-rating), but our $R$ has missing entries – scoring a blank as "0 stars" drags every factor towards zero
- Use specialized methods to find $P$, $Q$:

$$\min_{P,Q} \sum_{(x,i)\in R} \left(r_{xi} - q_i \cdot p_x\right)^2 \quad \text{where } \hat{r}_{xi} = q_i \cdot p_x$$

  the sum now runs over the *observed* entries only

- Note: cols of $P$, $Q$ need not be orthogonal/unit length ($\rightarrow$ strictly, no longer an SVD)
- $P$, $Q$ map users/movies to a latent space; the most popular model among Netflix contestants

# Overfitting Issues

- Want to minimize SSE for unseen test data
- *Idea*: Minimize SSE on training data
  - Want large $k$ (# of factors) to capture all the signals
  - But, SSE on test data begins to rise beyond a modest $k$, even while training SSE keeps falling
- This is a classical example of overfitting:
  - With too much freedom (too many free parameters) the model starts fitting noise
    - ✓ I.e., it fits too well the training data and thus not generalizing well to unseen test data
- The cure is not simply "use a small $k$": the next slide keeps $k$ large and *penalizes* large factor values instead

Toronto Metropolitan University

# REGULARIZATION

- To solve overfitting, we introduce regularization:
  - Allow rich model where there are sufficient data
  - Shrink aggressively where data are scarce

$$\min_{P,Q} \sum_{training} (r_{xi} - q_i p_x)^2 + \left[\lambda_1 \sum_x ||p_x||^2 + \lambda_2 \sum_i ||q_i||^2\right]$$

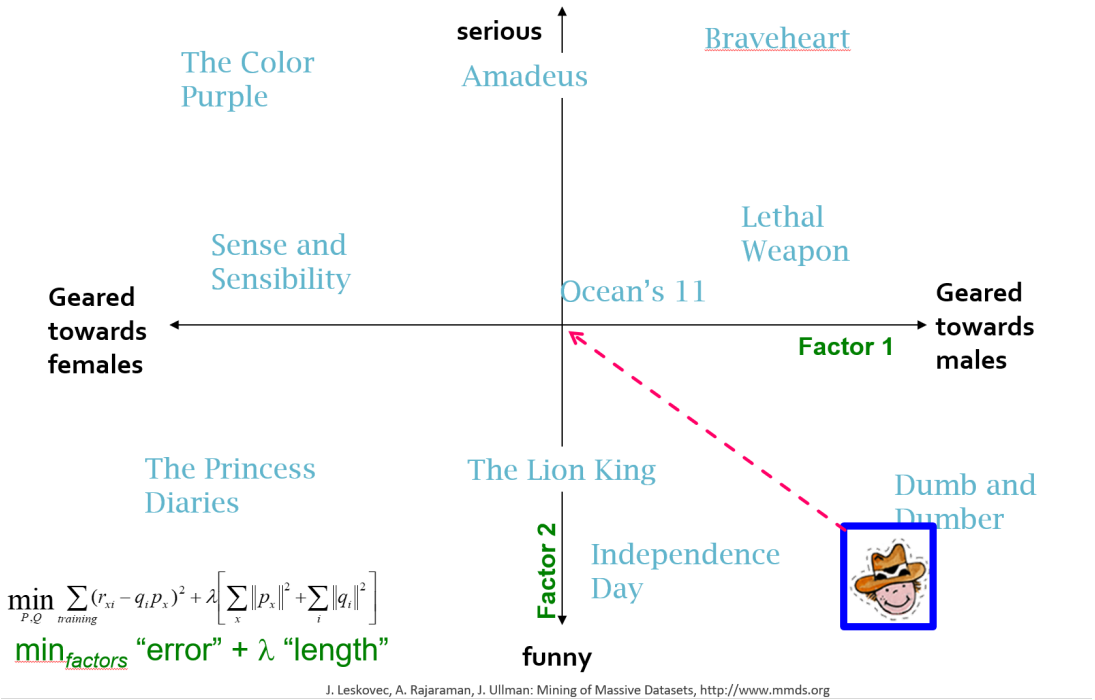
  - $\sum_{training}(r_{xi} - q_i p_x)^2$: error $\quad \lambda_1 \sum_x ||p_x||^2 + \lambda_2 \sum_i ||q_i||^2$: length

$$\min_{factors} error + \lambda \cdot length$$

  - $\lambda_1$, $\lambda_2$: user set *regularization* parameters ($\lambda_1$ on users, $\lambda_2$ on items), tuned on a validation set

    *Note*: We do not care about the "raw" value of the objective function, but we care about the $P, Q$ that achieve the minimum of the objective

# EFFECT OF REGULARIZATION




- The dashed arrow is the user vector $p_x$ being pulled *towards the origin* by the penalty $\lambda_1 ||p_x||^2$
- A user seen through few ratings is thereby stopped from claiming an extreme taste, and predictions for them stay near the baseline
- Users and items with plenty of evidence pay the same penalty, but their error term outweighs it and they stay far from the origin

# Gradient Descent

- Want to find matrices $P$ and $Q$:

$$\min_{P,Q} \sum_{training} (r_{xi} - q_i p_x)^2 + \left[\lambda_1 \sum_x ||p_x||^2 + \lambda_2 \sum_i ||q_i||^2\right]$$

- *Gradient descent*:
  - Initialize $P$ and $Q$ (using SVD, pretend missing ratings are 0)
    crude but cheap; the objective is not jointly convex in $(P, Q)$, so the starting point matters
  - Do gradient descent:
    - ✓ $P \leftarrow P - \eta \nabla P$, $\quad Q \leftarrow Q - \eta \nabla Q$ ($\eta$: learning rate)

      where $\nabla Q$ is gradient/derivative of matrix $Q$:

      $\nabla Q = [\nabla q_{if}]$ and $\nabla q_{if} = \sum_{x\,:\,(x,i)\in R} -2(r_{xi} - q_i p_x)p_{xf} + 2\lambda_2 q_{if}$

      Here $q_{if}$ is entry $f$ of row $q_i$ of matrix $Q$

- *Observation*: Computing gradients is slow! One update touches all $10^8$ ratings

# Stochastic Gradient Descent

- *Idea*: the GD gradient is a *sum of per-rating terms*; instead of evaluating it over all ratings, evaluate it for each individual rating and make a step

  GD: $Q \leftarrow Q - \eta\left[\sum_{r_{xi}} \nabla Q(r_{xi})\right]$ SGD: $Q \leftarrow Q - \eta \nabla Q(r_{xi})$

  $\Rightarrow$ Faster convergence in wall-clock time: need more steps, but each step is computed much faster

Algorithmic steps:

- Initialize $P$ and $Q$ (using SVD, pretend missing ratings are 0)
- Iterate over the ratings (multiple times if necessary) and update factors

  For each $r_{xi}$ (where $\eta_1, \eta_2$: learning rates):

  - $\xi_{xi} = 2(r_{xi} - q_i \cdot p_x)$ $\rightarrow$ derivative of the error
  - $q_i \leftarrow q_i + \eta_1(\xi_{xi} p_x - \lambda_2 q_i)$ $\rightarrow$ update equation
  - $p_x \leftarrow p_x + \eta_2(\xi_{xi} q_i - \lambda_1 p_x)$ $\rightarrow$ update equation
  - note the plus signs: $\xi_{xi}$ already carries the minus from $-\partial/\partial q_i$; constant factors of 2 are absorbed into $\lambda_1, \lambda_2$; both updates use the *old* $q_i, p_x$

Toronto Metropolitan University

# INCORPORATING BIASES INTO LATENT FACTOR MODEL

- *Baseline Predictor*: We have expectations on the rating by user $x$ of movie $i$, even without estimating $x$'s attitude towards movies like $i$
- Prediction with bias included:

$$\hat{r}_{xi} = \mu + b_x + b_i + q_i \cdot p_x$$

  - $\mu$: Overall mean rating; $b_x$: Bias for user $x$; $b_i$: Bias for movie $i$
  - $q_i \cdot p_x$: User-Movie interaction
  - same $\mu + b_x + b_i$ baseline as in "CF: Common Practice", but the biases are now *learned*

- *Example*:
  - Mean rating: 3.7
  - You are a critical reviewer: your ratings are 1 star lower than mean: $b_x = -1$
  - Star Wars gets a mean rating of 0.5 higher than average movie: $b_i = +0.5$
  - Predicted rating for you on Star Wars: $= 3.7 - 1 + 0.5 = 3.2$ (before the $q_i \cdot p_x$ correction)

Toronto Metropolitan University

# Fitting the New Model

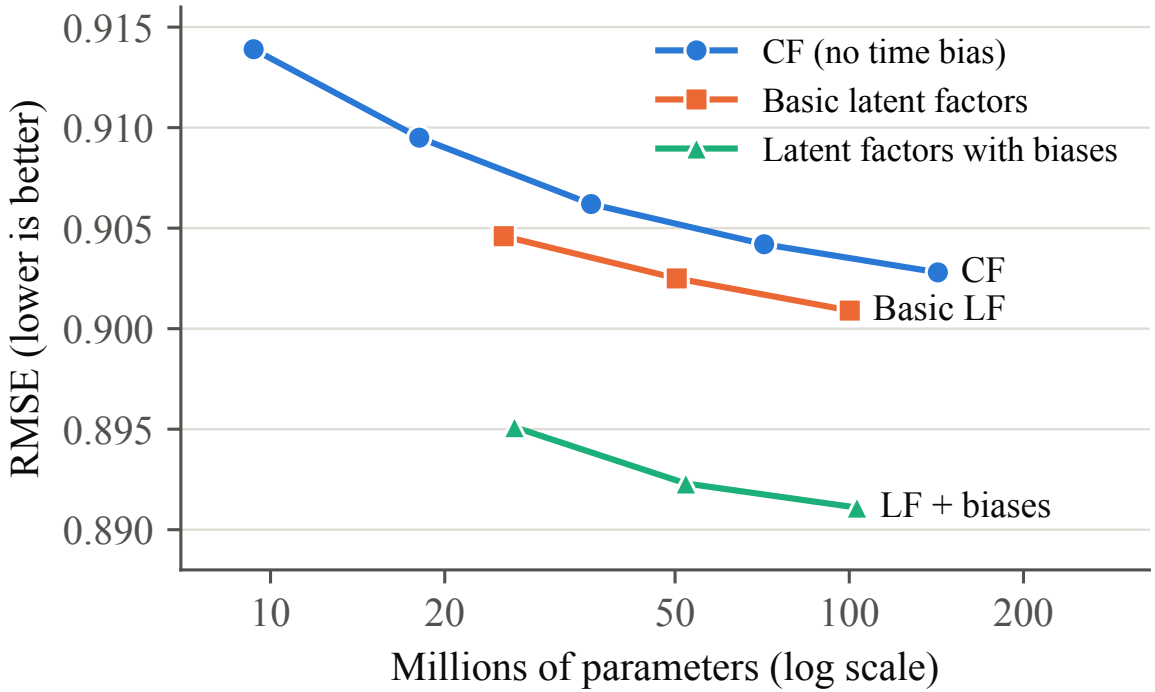
- Solve:

$$\min_{Q,P,b_x,b_i} \sum_{(x,i)\in R} \left(r_{xi} - (\mu + b_x + b_i + q_i \cdot p_x)\right)^2 + \left(\lambda_2 \sum_i ||q_i||^2 + \lambda_1 \sum_x ||p_x||^2 + \lambda_3 \sum_x b_x^2 + \lambda_4 \sum_i b_i^2\right)$$

  $\lambda_1$ on users and $\lambda_2$ on items; $\lambda_3, \lambda_4$ on the user and item biases

- Use stochastic gradient descent to find parameters
- *Note*: Both biases $b_x,\ b_i$ as well as interactions $q_i,\ p_x$ are treated as parameters (i.e., we estimate them)
- The four $\lambda$'s are tuned on a validation set

# Performance of Various Methods




Netflix RMSE against model size. Two lessons: latent factors beat neighborhood CF at equal size, and *adding the biases $b_x, b_i$ buys more than a threefold increase in parameters does* – latent factors with biases at about 26M parameters (RMSE 0.895) beat basic latent factors at 100M (0.901). Cinematch scored 0.9514 and the prize required 0.8563 (quiz-half figures), so no single method here wins on its own.

Re-plotted from the figure in Mining of Massive Datasets *by* Leskovec, Rajaraman, Ullman